Iterative LLM-Based NAS with Feedback Memory

Anonymous CVPR submission

Paper ID

Abstract

Neural Architecture Search (NAS) automates network design but typically demands substantial computational resources. We propose a closed-loop pipeline that leverages large language models (LLMs) to iteratively generate, evaluate, and refine convolutional neural network architectures for image classification. Central to our approach is a historical feedback memory *mechanism inspired by Markov chains: a sliding window of the most recent improvement attempts is fed back to the LLM, enabling it to learn from past successes and failures without additional training. Using one-epoch proxy accuracy on CIFAR-10 [?] and ImageNette [?] as fast architecture ranking signals, we evaluate the pipeline with three instruction-tuned LLMs across up to 2000 iterations. On CIFAR-10, DeepSeek-Coder-6.7B-Instruct improves generated architectures from 28.2% to a peak of 69.2% over 1519 successful evaluations, Qwen2.5-7B-Instruct improves from 50.0% to 71.5% over 376 successful evaluations, and GLM-5 improves from 55.1% to 68.1% over 86 successful evaluations. On ImageNette, Qwen2.5-7B-Instruct improves from 23.0% to 54.6% over 642 successful evaluations, and GLM-5 improves from 41.1% to 58.3% over 81 successful evaluations. All models exhibit statistically significant upward trends on both datasets. Compared to prior iterative LLM optimization methods that rely on global elite retention (discarding failure trajectories) [? ?], require complex population maintenance [?], or operate in constrained discrete search spaces [?], our bounded Markovian memory explicitly models sequential code execution failures in an open code space. This structured causal learning from recent mistakes is a critical requirement for neural architecture generation that distinguishes our approach from existing work. These results establish that LLMs of varied origins and specializations can serve as effective architecture designers when equipped with iterative feedback and historical memory, offering a lightweight and accessible paradigm for LLM-driven NAS.*

1. Introduction

Neural Architecture Search (NAS) has emerged as a powerful paradigm for automating the design of deep neural networks, achieving competitive or superior performance to hand-crafted architectures across diverse tasks [? ? ?]. However, conventional NAS methods—whether based on reinforcement learning [?], evolutionary algorithms [?], or differentiable relaxations [?]—remain computationally expensive, often requiring hundreds or thousands of GPU hours to search a single architecture. This cost has motivated training-free approaches that use proxy metrics to rank architectures without full training [? ?], yet these methods still operate within predefined, discrete search spaces.

Recent advances in large language models (LLMs) have opened a fundamentally different avenue: using LLMs as architecture generators that produce executable neural network code directly [? ? ?]. Unlike traditional NAS, LLM-based approaches operate in the space of *programs* rather than fixed cell-based encodings, enabling more flexible and expressive architecture design. Prior work within the NNGPT framework [?] and related efforts [? ? ?] have demonstrated that LLMs can generate functional vision models, but predominantly through single-shot or few-shot generation without iterative self-improvement.

A critical limitation of single-shot LLM generation is that it treats architecture design as a one-pass prediction, discarding the evaluation signal entirely. In contrast, human engineers iterate: they build, test, analyze failures, and refine their designs based on accumulated experience. This observation motivates our central question: *Can a small LLM iteratively improve neural network architectures by learning from a structured history of its own attempts?*

We propose a closed-loop pipeline comprising three components: (1) a **Code Generator** that prompts an instruction-tuned LLM to produce PyTorch model implementations, (2) an **Evaluator** that trains each generated model for a single epoch on CIFAR-10 as a fast proxy for architecture quality, and (3) a **Prompt Improver** that analyzes evaluation results alongside a sliding window of recent iteration history—a mechanism we term *historical*

feedback memory—to produce targeted improvement suggestions for the next iteration. This design draws on the Markov property: the improvement decision at each step depends on the current best architecture and a bounded window of recent transitions, rather than the full trajectory.

Our main contributions are:

- A closed-loop, iterative NAS pipeline driven by LLMs that progressively discovers better architectures through code generation, evaluation, and prompt refinement.
- A *historical feedback memory* mechanism that maintains a sliding window of past improvement attempts, enabling the LLM to mitigate repeating failed strategies and build on successful ones.
- An empirical demonstration across three LLMs of different origins and specializations on two datasets (CIFAR-10 and ImageNette) showing that the pipeline consistently improves architecture quality—e.g., from 50.0% to 71.5% on CIFAR-10 and from 23.0% to 54.6% on ImageNette with Qwen2.5-7B.

2. Related Work

Neural Architecture Search. NAS methods aim to automate the design of neural network architectures. Early approaches employed reinforcement learning [?] and evolutionary algorithms [?] to explore architecture search spaces, but required thousands of GPU hours per search. Parameter-sharing methods such as ENAS [?] and differentiable approaches like DARTS [?] substantially reduced search cost by amortizing evaluation across architectures. More recently, *zero-shot* or *training-free* NAS methods [? ?] bypass training entirely by scoring architectures at initialization using proxy metrics such as activation overlap [?] or gradient-based indicators. While effective within their respective search spaces, these methods remain constrained to predefined, discrete architecture parameterizations (e.g., cell-based structures) and cannot generate truly novel architectural patterns.

LLMs for Code Generation and AutoML. Large language models have demonstrated strong capabilities in code generation [?], and their application to machine learning automation is an active research area. The NNGPT framework [?] pioneered the use of LLMs as neural network generators, showing that language models can produce functional vision models from textual prompts. Subsequent work explored fractal-inspired architectures [?], few-shot prompting strategies [?], novel architecture creativity [?], and non-standard channel priors [?]. LLMs have also been applied to hyperparameter optimization [?], data transformation design [?], and collaborative vision-language pipelines [? ? ?]. These efforts are supported by curated datasets of neural network architectures [? ?] and deployment pipelines [?]. However, most existing approaches treat LLM-based architecture generation as a single-step process, without leveraging iterative refinement from evaluation feedback. Our work extends this line by introducing a closed-loop pipeline with historical memory that enables progressive architecture improvement.

Iterative LLM Optimization. Recent work has explored using LLMs as iterative optimizers that progressively improve solutions by learning from historical attempts. OPRO [?] demonstrated that LLMs can optimize text prompts and discrete solutions by conditioning on sorted history of past candidates and scores. FunSearch [?] extended this to program synthesis for mathematical discovery using island-based evolutionary search. EvoPrompting [?] applied evolutionary prompting with LLMs as mutation operators for code-level neural architecture search. ReEvo [?] introduced dual-layer reflective memory for LLM-driven heuristic algorithm design. More directly related to NAS, LLMO [?] used LLMs as combinatorial optimizers within the predefined cell-based search space of NAS-Bench-201, replacing traditional meta-heuristics. However, existing methods face key limitations: reliance on global elite histories that discard crucial failure trajectories [? ?], the computational overhead of maintaining evolutionary populations [?], operation in constrained discrete search spaces [?], or targeting domains without structural code constraints [?]. Our work introduces bounded Markovian memory specifically designed for iterative neural architecture code generation in open search spaces, with explicit failure modeling enabling effectiveness on small frozen LLMs.

3. Methodology

Inspired by recent advancements in the application of LLMs across various domains [? ? ? ?] and prior architectural synthesis experiments within the NNGPT framework [? ? ? ? ? ? ?], and leveraging the existing LEMUR dataset of a broad range of high-capacity and edge-optimized models [? ?], we developed an iterative NAS pipeline based on the NNGPT framework [?]. The pipeline consists of three core modules operating in a closed loop, as illustrated in Figure ??.

3.1. Pipeline Overview

At each iteration t, the pipeline executes the following cycle: (1) the Code Generator produces a candidate architecture $\mathcal{A}_t$ as executable PyTorch code; (2) the generated code is validated and, if structurally correct, trained and evaluated by the Evaluator to obtain a proxy accuracy a_t; (3) the Prompt Improver analyzes the result in the context of the current best architecture and recent iteration history, producing improvement suggestions s_t for the next iteration.

The best-performing architecture $\mathcal{A}^* = \arg\max_{i \le t} a_i$ is maintained as the reference implementation throughout the search.

3.2. Code Generator

The Code Generator uses a pre-trained, instruction-tuned LLM to produce a complete PyTorch model class implementing `nn.Module`. The LLM receives a fixed prompt template comprising: (i) a role description positioning the model as a "visionary deep learning architect," (ii) the task specification (CIFAR-10 image classification without pre-trained weights), (iii) the current best implementation $\mathcal{A}^*$ as reference code, and (iv) the improvement suggestions s_{t-1} from the previous iteration. The generated code must define a `Net(nn.Module)` class with standard `__init__` and `forward` methods. Generation uses temperature $\tau = 0.7$ and nucleus sampling ($p = 0.9$) to balance architectural diversity with code coherence.

3.3. Evaluator

Generated code undergoes a two-stage evaluation process.

Quick validation. The model is instantiated and a forward pass is performed with a dummy input tensor of shape $2 \times 3 \times 32 \times 32$. The output shape is verified to match $B \times 10$ (for batch size B and 10 CIFAR-10 classes). Models failing this check are discarded with an error message forwarded to the Prompt Improver.

Proxy training. Models passing validation are trained for **one epoch** on the target dataset (CIFAR-10 or ImageNette) using SGD (momentum 0.9, weight decay 5×10^{-4}), an initial learning rate of 0.01 with cosine annealing, and a batch size of 128. Standard data augmentation is applied: random crop with 4-pixel padding and random horizontal flip. Top-1 test accuracy on the target test/validation set serves as the proxy metric. Training is executed in an isolated subprocess with a 30-minute timeout. This one-epoch accuracy provides a fast, informative signal for ranking architectures [?], enabling rapid iteration without the cost of full convergence training.

3.4. Prompt Improver with Historical Feedback Memory

The Prompt Improver is the central component enabling iterative learning. After each evaluation, it receives three inputs: (1) the best code $\mathcal{A}^*$ and its accuracy a^*, (2) the current iteration's code $\mathcal{A}_t$ and its evaluation outcome (accuracy value or error message), and (3) a *historical feedback memory* $\mathcal{H}_t$ containing the last K=5 improvement attempts.

Each history entry $h_i \in \mathcal{H}_t$ is a triple recording: the identified problem, the suggested improvement, and the resulting outcome (accuracy achieved or error encountered). This sliding-window design draws on the Markov property: the improvement decision at step t depends on the current state (best architecture plus recent history window) rather than the complete trajectory. The bounded window prevents context overflow while providing sufficient signal for the LLM to identify recurring failure patterns.

Markov Property Formalization. Let $\mathcal{A}_t$ denote the architecture generated at step t, a_t its evaluation outcome (accuracy or error), and s_t the improvement suggestions. Our historical feedback memory $\mathcal{H}_t^{(K)}$ maintains exactly the most recent K=5 improvement attempts:

$$\mathcal{H}_t^{(K)} = \{(s_{t-K}, a_{t-K}), \ldots, (s_{t-1}, a_{t-1})\}. \quad (1)$$

The improvement suggestion generation satisfies the **K-order Markov property**:

$$P(s_t \mid \mathcal{A}^*, \mathcal{H}_t) = P(s_t \mid \mathcal{A}^*, \mathcal{H}_t^{(K)}), \quad (2)$$

meaning the next suggestion depends only on the current best architecture and the bounded recent history, not the complete trajectory. This design ensures **constant context size** and avoids the context overflow problem faced by unbounded history approaches like OPRO [?].

Each history entry (s_i, a_i) is structured as a **diagnostic triple**:

$$s_i = (\text{problem}_i, \text{suggestion}_i, \text{outcome}_i), \quad (3)$$

where problem_i identifies the architectural deficiency, suggestion_i proposes concrete code modifications, and outcome_i records whether the suggestion succeeded (accuracy gain) or failed (error type). This structured format enables the LLM to learn causal patterns between design decisions and outcomes, unlike scalar-score histories in prior work [? ?].

The Prompt Improver produces a structured response containing: (a) a **reason** diagnosing the current result, (b) an **inspiration** drawing on cross-disciplinary insights, and (c) concrete **improvement suggestions** for the next iteration. These suggestions, together with the best code, form the input to the Code Generator in the subsequent iteration, closing the feedback loop.

4. Experiments

4.1. Experimental Setup

Datasets. We evaluate our approach on CIFAR-10 [?] and ImageNette [?]. CIFAR-10 comprises 60,000 32×32 color images across 10 classes (50,000 training, 10,000 test) and operates with standard augmentation (random crop, horizontal flip). ImageNette is a 10-class subset of ImageNet containing higher-resolution images, resoluted here to 160×160, presenting a notably more challenging architectural design problem for the models. Standard data augmentations were kept commensurate across iterations.

Algorithm 1: Iterative LLM-Driven NAS Pipeline

Input: LLM $\mathcal{L}$, max iterations T, history window K

$\mathcal{A}^* \leftarrow \emptyset$; $a^* \leftarrow 0$; $\mathcal{H} \leftarrow \emptyset$; $s_0 \leftarrow \emptyset$;
for $t \leftarrow 1$ **to** T **do**
  $\mathcal{A}_t \leftarrow \text{Generate}(\mathcal{L}, \mathcal{A}^*, s_{t-1})$;
  if *Validate*$(\mathcal{A}_t)$ *succeeds* **then**
    $a_t \leftarrow \text{TrainAndEvaluate}(\mathcal{A}_t)$;
  else
    $a_t \leftarrow$ error message;
  if $a_t > a^*$ **then**
    $\mathcal{A}^* \leftarrow \mathcal{A}_t$; $a^* \leftarrow a_t$;
  Append (s_{t-1}, a_t) to $\mathcal{H}$; keep last K entries;
  $s_t \leftarrow \text{Improve}(\mathcal{L}, \mathcal{A}^*, \mathcal{A}_t, a_t, \mathcal{H})$;
return $\mathcal{A}^*$, a^*

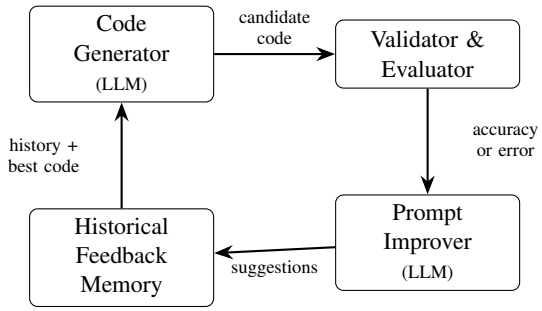


Figure 1. Overview of the iterative NAS pipeline. The Code Generator produces candidate architectures as executable PyTorch code. The Evaluator validates and trains them using one-epoch proxy evaluation. The Prompt Improver analyzes results with historical feedback memory to generate targeted improvement suggestions for the next iteration.

Language models. To evaluate the pipeline across model specializations, we run separate experiments with three instruction-tuned LLMs: code-specialized DeepSeek-Coder-6.7B-Instruct, small generalist Qwen2.5-7B-Instruct, and large generalist GLM-5. All models use the same generation parameters: temperature 0.7, nucleus sampling $p = 0.9$, maximum 2,048 new tokens. A fixed seed with incremental call counter ensures reproducible but diverse generation across iterations. DeepSeek-Coder-6.7B and Qwen2.5-7B are each run for 2000 iterations; GLM-5 is only run for 100 iterations for economic reasons.

Training protocol. Each candidate architecture is trained for one epoch with SGD (momentum 0.9, weight decay 5×10^{-4}), learning rate 0.01 with cosine annealing, and batch size 128. A fixed random seed (43) and deterministic CUDA operations ensure reproducibility. Training runs in isolated subprocesses with a 30-minute timeout. Fine-tuning of LLMs and training of computer vision models are performed on NVIDIA GeForce RTX 4090 24G GPU.

Baseline. We compare against **single-shot generation**: the accuracy of the first successfully generated and evaluated architecture (iteration 1), representing what the LLM produces from its pre-trained knowledge alone without any feedback signal.

4.2. Evaluation Metrics

To evaluate our approach rigorously, we adopt true validation accuracy after the first training epoch as our primary architecture quality metric [**?**], rather than relying on zero-shot NAS proxies. Although zero-shot proxies are correlated with fully trained accuracy [**? ? ?**], they remain indirect indicators of performance. Even the strongest reported Spearman rank-correlation coefficients ($\rho \approx 0.5$–0.82) on standard benchmarks such as NAS-Bench-101 and NAS-Bench-201 correspond to a coefficient of determination of at most $R^2 \approx 0.67$, leaving substantial variance unexplained [**? ? ? ?**]. By using first-epoch validation accuracy instead, we aim to demonstrate more directly that our algorithm can reliably influence and accelerate early-stage performance trajectories of neural networks.

To characterize search dynamics, we compute: Spearman rank correlation ρ between iteration index and accuracy (trend strength), and descriptive statistics (mean $\pm$ SD) for the first and last 5 successful evaluations. All trends are evaluated for statistical significance.

4.3. Main Results

Table **??** compares the key results for all three LLMs across CIFAR-10 and ImageNette. Almost all models exhibit statistically significant upward trends in architecture quality on both datasets (Spearman $\rho > 0.56$, $p \approx 0$), confirming that the iterative pipeline with historical feedback memory consistently drives improvement regardless of model origin, specialization, or dataset complexity.

DeepSeek-Coder-6.7B-Instruct. On CIFAR-10, the code-specialized model achieved 1519 successful evaluations (76% success rate) and improved from 28.2% to a peak of 69.2% ($\rho = 0.754$). Conversely, on ImageNette, DeepSeek struggled significantly, successfully producing only 13 valid architectures across 2000 iterations (0.6% success rate), which hindered any statistically meaningful progress.

Qwen2.5-7B-Instruct. The generalist model successfully improved on both CIFAR-10 (from 50.0% to 71.5%, $\rho = 0.561$) and ImageNette (from 23.0% to 54.6%, $\rho = 0.663$). Although it encountered a high error rate on CIFAR-10

(19% success rate), it demonstrated greater robustness on ImageNette (32% success rate) with steady progressive optimization.

GLM-5. The generalist model from ZhipuAI consistently maintained a high success rate on both CIFAR-10 (86%) and ImageNette (81%). Within merely 100 iterations, it rapidly converged to stable architectures, improving accuracy from 55.1% to 68.1% ($\rho = 0.720$) on CIFAR-10 and from 41.1% to 58.3% ($\rho = 0.631$) on ImageNette.

Figure **??** shows the accuracy trajectories for all models. All three curves display consistent upward trends: DeepSeek-Coder achieves the largest absolute improvement (+41.0 pp from single-shot to peak), Qwen2.5 reaches the highest peak accuracy (71.5%), and GLM-5 demonstrates the fastest initial convergence rate among its 100 iterations.

5. Discussion

Effectiveness of iterative refinement. All models exhibit statistically significant positive correlations between evaluation index and accuracy ($\rho \in [0.56, 0.75]$, all $p \approx 0$), confirming that historical feedback memory enables systematic architectural improvement across different LLM origins and specializations. Access to the last K=5 improvement attempts provides the LLM with sufficient context to identify recurring failure patterns and avoid previously unsuccessful strategies. All accuracy curves display characteristic increasing behavior—rapid early gains followed by gradual flattening—mirroring patterns in traditional optimization and suggesting that the LLM-driven search exhibits meaningful optimization dynamics rather than random exploration.

Effect of model specialization and iteration scale. The three models reveal distinct behavior along two axes. DeepSeek-Coder-6.7B (code-specialized) achieves the largest absolute improvement (+41.0 pp from single-shot to peak) with a high and stable success rate (76%) over 2000 iterations, suggesting that code specialization yields both reliable generation and systematic progression. Qwen2.5-7B (generalist) achieves the highest peak accuracy (71.5%) at the cost of a dramatically lower success rate (19%) over 2000 iterations—its Markovian exploration increasingly targets complex, ambitious architectures that more often fail validation but, when successful, surpass DeepSeek-Coder's best. GLM-5 (generalist, ZhipuAI) demonstrates rapid early convergence (peak at eval #24, +13.0 pp improvement), reaching stable high-quality architectures within 100 iterations, making it the most iteration-efficient of the three.

Comparison with iterative LLM optimization methods. Our approach shares conceptual similarities with recent work on using LLMs as iterative optimizers [**?** **?** **?**], but differs in three critical dimensions that enable its effectiveness on neural architecture search:

Bounded Markovian memory. Unlike OPRO [**?**] or FunSearch [**?**], which populate prompts with historically global top-performing solutions (global elite retention), our sliding window maintains strictly the last K=5 sequential attempts. In code-level NAS, the architecture's structure evolves rapidly; therefore, ancient successes are less informative than immediate recent failures. The Markov property enforces that the LLM focuses entirely on local causal relationships (what modification just caused what specific error), avoiding the distraction of irrelevant global history.

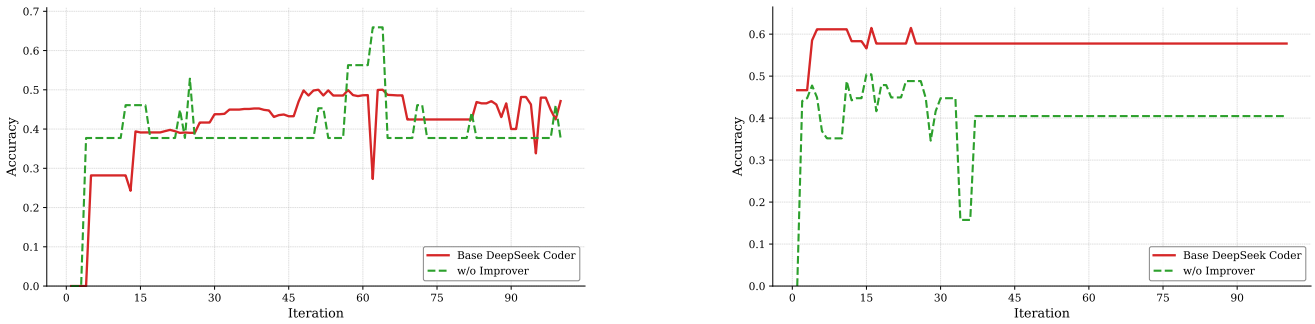
Structured failure modeling. Our diagnostic triples explicitly encode code execution failures (tensor shape mismatches, `AttributeErrors` from undefined modules, syntax errors) as first-class entries in the history, enabling the LLM to learn from structural mistakes. This is fundamentally different from prior work where low-performing solutions are simply discarded from the prompt [**?** **?**] or managed implicitly through population selection [**?**]. As shown in Section **??**, failure rates of 23–59% across our runs highlight that code generation for neural architectures is inherently error-prone; ignoring these failures would discard critical learning signals.

Small-model viability. Our results demonstrate that ≤7B LLMs of diverse origins (DeepSeek AI, Alibaba Cloud) can effectively perform NAS when equipped with structured feedback (all $\rho \geq 0.56$, all $p \approx 0$), whereas prior iterative LLM optimization work primarily relies on large models (OPRO uses GPT-4 and PaLM 2-L [**?**]). We hypothesize this is enabled by our dual-LLM specialization: the Code Generator focuses solely on code synthesis given structured suggestions, while the Prompt Improver handles diagnostic reasoning. This task decomposition reduces per-call cognitive load compared to end-to-end generation [**?**].

Table **??** positions our method relative to the five most similar LLM-based NAS and optimization approaches. We are the only method that simultaneously satisfies all five properties: (1) targeting neural architecture search, (2) operating in an unconstrained code search space rather than predefined cells, (3) automated failure diagnosis via structured diagnostic triples, (4) demonstrated effectiveness on a single consumer GPU with ≤7B LLMs, and (5) requiring no LLM fine-tuning whatsoever. Notably, while EvoPrompting also performs code-level NAS on small models, it requires soft prompt-tuning of the LLM, whereas our pipeline operates on fully frozen instruction-tuned models. LLMO also applies LLMs to NAS without training, but operates in predefined discrete search spaces that fundamentally limit architectural expressiveness.

Table 1. Comparison of the iterative NAS pipeline on CIFAR-10 and ImageNette (one-epoch proxy accuracy) across three LLMs. Models generally show statistically significant improvement over single-shot generation ($p \approx 0$). DeepSeek on ImageNette is an outlier due to extremely low validation success rate.

	CIFAR-10			ImageNette		
Metric	**DeepSeek**	**Qwen2.5**	**GLM-5**	**DeepSeek**	**Qwen2.5**	**GLM-5**
Total iterations	2000	2000	100	2000	2000	100
Successful evaluations	1519	376	86	13	642	81
Success rate	76%	19%	86%	0.6%	32%	81%
Single-shot accuracy	0.282	0.500	0.551	0.467	0.230	0.411
Best accuracy	0.692	0.715	0.681	0.615	0.546	0.583
First 5 evals (mean ± SD)	0.282 ± 0.0	0.492 ± 0.030	0.512 ± 0.041	0.467 ± 0.0	0.230 ± 0.0	0.411 ± 0.0
Last 5 evals (mean ± SD)	0.670 ± 0.011	0.672 ± 0.029	0.681 ± 0.000	0.578 ± 0.031	0.440 ± 0.041	0.583 ± 0.0
Improvement	+0.388	+0.180	+0.169	+0.111	+0.210	+0.172
Spearman ρ	0.754	0.561	0.720	−0.084	0.663	0.631



(a) First 100 iterations of DeepSeek-Coder 6.7B on CIFAR-10

(b) First 100 iterations of DeepSeek-Coder 6.7B on ImageNette

Figure 2. Ablation study comparing the full pipeline (Base) against a baseline without the Prompt Improver (w/o Improver) over 100 iterations. Errors encountered before the first successful evaluation are plotted at 0%, while subsequent errors fall back to the most recent valid accuracy. Without the Prompt Improver, DeepSeek-Coder suffers severe programmatic instability on CIFAR-10 (a) and completely fails to generate any valid networks on ImageNette (b) beyond its initial shots. The iterative evolutionary feedback is crucial for structural coherence.

One-epoch proxy as architecture ranking signal. Using one-epoch accuracy as a ranking signal offers a pragmatic trade-off between evaluation cost and informativeness. While the absolute accuracy values (10–59%) are well below state-of-the-art CIFAR-10 results with full training (exceeding 95%), prior work on training-free NAS [**? ?**] has established that cheap proxy metrics can effectively rank architectures. Our one-epoch proxy serves a similar role: it distinguishes between architectures of varying quality at minimal cost, enabling the feedback loop to guide the LLM toward better designs. Retraining the best-discovered architectures with full training schedules could verify whether the one-epoch ranking transfers to converged performance.

Limitations. Our study has several limitations. First, the code generation success rates vary substantially (19–86%), and the causes of Qwen2.5's high failure rate at scale (19% over 2000 iterations) merit further investigation into prompt design and constrained decoding strategies. Second, our evaluation is limited to CIFAR-10; experiments on larger-scale benchmarks and diverse tasks would better demonstrate versatility.

Future work. Promising directions include: (1) retraining the top-K discovered architectures with full training schedules to verify their converged performance; (2) extending the cross-model comparison to additional LLM families and sizes; (3) integrating zero-shot proxy metrics [**? ?**] as complementary evaluation signals to further accelerate the search; (4) extending to larger datasets (e.g., ImageNet) and tasks beyond image classification; and (5) apply genetic algorithm for historical feedback memory.

6. Conclusion

We presented an iterative NAS pipeline that leverages instruction-tuned LLMs to progressively generate and improve neural network architectures through a closed loop

of code generation, evaluation, and prompt refinement with historical feedback memory. Evaluated on CIFAR-10 with three LLMs of different origins and specializations across up to 2000 iterations, the pipeline significantly improved one-epoch proxy accuracy across all models—from 28.2% to 69.2% with DeepSeek-Coder-6.7B on CIFAR-10, from 50.0% to 71.5% with Qwen2.5-7B on CIFAR-10, and from 23.0% to 54.6% with Qwen2.5-7B on ImageNette—demonstrating that iterative feedback with bounded historical memory significantly outperforms single-shot generation across model origins and specializations. Three design principles distinguish our approach from prior iterative LLM optimization methods: (1) operating in an open code space rather than constrained predefined cells, (2) bounded K-step Markovian memory prevents context overflow while retaining relevant failure patterns, and (3) structured diagnostic triples explicitly model code execution failures, which alongside dual-LLM specialization (Code Generator and Prompt Improver) enables effective operation on small frozen models without fine-tuning. Our results establish that even $\leq$7B LLMs from diverse AI ecosystems can serve as effective architecture search agents when equipped with structured iterative feedback, offering a lightweight and accessible approach to neural architecture search.